



CHALLENGE 13 / 20

THE HOSPITAL IMAGE ARCHIVE BREACH

PRIVACY-PRESERVING BREACH SCOPING IN MEDICAL IMAGING SYSTEMS

NATIONAL LEVEL HACKATHON • 2026

POWERED BY SYNTHOQUEST PVT LTD

Student Project Development Challenge • No organizer-provided evidence package.

1 | PROBLEM BACKGROUND

A hospital group with 38 hospitals and 11 million patients holds about 900 million images in its imaging archives, along with a de-identification pipeline for research and links to teleradiology partners. A vendor maintenance account was used for 27 days from three address ranges. The vendor also legitimately performs bulk integrity scans, migrations and AI-model integration pulls, so its activity looks routine.

Logs record access at study level. Some archive nodes log only hourly aggregates, and one node had logging disabled for five days. Image metadata carries names and identifiers, and some images have patient identifiers burned into the pixels. Partner sites hold cached copies.

Data protection law requires notifying affected patients. The group cannot notify 11 million, cannot under-notify, and must not expose patient data to its own investigators beyond what is necessary.

2 | PROBLEM STATEMENT

Develop a **Patient-Level Breach Scoping System** that estimates, for each patient, the probability that identifiable data was exfiltrated, groups patients into affected, possibly affected and unaffected with defensible bounds, and separates malicious use of the vendor account from the vendor's legitimate bulk operations.

The system must be private by design: investigators must never see patient-identifiable data beyond what notification requires, and this property must be demonstrable.

Hard constraints

- Nine hundred million objects, aggregated logs and a period with no logs.
- Legitimate bulk operations look like exfiltration and the reverse.
- Burned-in identifiers and cached partner copies widen the exposure.
- Privacy protection must be shown by the design and tested, not asserted.
- The notification list must be justified, with over-notification and under-notification both measured.

No evidence package or dataset is provided. Teams decide what to build, what data they need, how to simulate the environment and how to validate the result. No tools or architecture are prescribed.

3 | MANDATORY FEATURES

Teams must develop a working software solution. The system must provide the following features. How each is achieved is for the team to decide.

01	Coarse-Log Exposure Inference	Ability to infer patient-level exposure from aggregated and missing logs.
02	Vendor Activity Separation	Ability to separate malicious use from legitimate bulk operations.
03	Logging-Gap Bounding	Ability to bound exposure during periods with no logs.
04	Indirect Exposure Assessment	Ability to include burned-in identifiers and cached copies.
05	Patient Population Classification	Ability to classify patients as affected, possible or unaffected.
06	Privacy-by-Design Investigation	Ability to investigate without exposing patient data.
07	Notification Optimisation	Ability to minimise over- and under-notification.
08	Scale Handling	Ability to work at hundreds of millions of objects.

The system must clearly distinguish between:

AFFECTED	POSSIBLY AFFECTED	UNAFFECTED	ESTIMATED	UNKNOWN
----------	-------------------	------------	-----------	---------

A patient who cannot be shown as unaffected must not be presented as unaffected.

Build, don't search. This is a development challenge. Teams must create the system and the controlled environment that proves it works. Submitting an analysis of a given dataset is not a solution.

4 | EXPECTED OUTPUT

Live demonstration. The team must show that its system can handle a complex controlled scenario created by the team. The demonstration must show:

01	Teams' own simulated archive with synthetic patients and legitimate bulk operations
02	A period of disabled logging
03	Separation of vendor misuse from vendor routine
04	Patient-level classification
05	Demonstration of privacy properties
06	Over- and under-notification rates
07	Performance at scale

The team must be able to explain why the system reached each major conclusion.

Validation standard (every team)

- Build a scenario generator with known ground truth and evaluate on at least ten distinct scenarios, including ones with deliberately misleading evidence.
- At the demonstration, judges change scenario parameters on the spot. The system must adapt without being rebuilt.
- Include an adversary variant that actively tries to defeat the system, and report how the system held up.
- Stated confidence must match measured accuracy. Overconfident or unexplained conclusions lose marks.
- No real systems, real personal data or functional malware may be used.

Submission. Each team must submit:

01	Working Prototype	A functional implementation of the proposed solution.
02	Source Code	Complete source code required to reproduce the project.
03	Technical Architecture	Architecture and component description.
04	Validation Environment	A controlled environment created by the team to demonstrate the solution.
05	Technical Documentation	Implementation, design and testing documentation.
06	Forensic Investigation Report	A professional report generated from the demonstrated investigation.
07	Demonstration	Live technical demonstration to the judging panel.
08	Limitations	Known limitations and situations where the system cannot establish a reliable conclusion.